Doctors Reservation System For Patients

TEAM 5

Student Name	Student ID	SectionNo.
Xatamov Javohirbek	U2210251	004
Zikirillayev Asadtilla	U2210280	004
Sodikov Akmaljon	U2210214	004
Eshboyev Temur	U2210068	002
Sabitov Samandar	U2210197	004
Kobiljonov Davlatbek	U2210118	001

Outline

- Section 1: Problem Statement – Zikirillayev Asadtilla U2210280 004
- Section 2: Abstract – Sodikov Akmaljon U2210214 004
- Section 3: Introduction – Javohirbek Xatamov U2210251 004
- Section 4: Overall Project Overview – Kobiljonov Davlatbek U2210118 001
- Section 5: Requirement Definition – Sabitov Samandar U2210197 004
- Section 6: Project Design & Implementation – Zikirillayev Asadtilla U2210280
- Section 7: Results & Discussion – Eshboyev Temur U2210068 002
- Section 8: Conclusion – Javohirbek Xatamov U2210251 004
- Section 9: Future Work – Javohirbek Xatamov U2210251 004

Problem Statement

The Problem:

Patients face long waits, difficulty finding specialists, and lack of transparency in medical appointments in Uzbekistan, causing stress and frustration.

Who It Affects:

Patients: Long queues, wasted time, and struggle to find available specialists.
Doctors and Hospitals: Inefficient resource management, overcrowded waiting rooms, administrative overload.
Families: Stress and lost productivity.

Root Causes:

Outdated, manual appointment systems. Lack of digitization and centralized platforms. Limited accessibility to medical history.

Abstract

Doctors Reservation System for Patients (DRSP):

Project Overview: A Linux-based desktop application implemented in a client-server style to efficiently manage medical appointments. Core Modules:

Patient Management Module:

Register and select medical specialty. Book appointments with real-time availability.

Doctor Management Module:

Manage schedules and access patient histories. Dynamic waitlist for cancellations.

System Administrator:

Monitor system operations. Manage database records and user activities.

INTRODUCTION

Doctors Reservation System for Patients (DRSP):

Overview: Addressing challenges in patient-doctor interactions and appointment scheduling in healthcare.

User Types:

Patients:

Registration and appointment booking. Access to medical histories. Join waitlists and receive notifications for appointment status changes.

Doctors:

Manage daily schedules and patient appointments. Access relevant patient information. Accept/reject appointments and manage waitlists.

System Administrators:

Monitor system operations.Manage database records and user activities.

INTRODUCTION

Core Functionality:

1. Hierarchical hospital selection.
2. Real-time appointment scheduling.
3. Automated waitlist management.
4. Notification system.

Technical Implementation:

Frontend: Intuitive interfaces for patients and doctors.
Middle Layer: Data transfer between client and server.
Backend: Handles user authentication, hospital/doctor management, appointment scheduling, waitlist processing, and medical record management.

INTRODUCTION

Technical Requirements:

1. User authentication and authorization.
2. Database management for patient records and appointments.
3. Reliable communication between frontend and backend.
4. Maintain data integrity during concurrent bookings and updates.

Overall Project Overview

System Components Overview

1. Client-Side Components

a) Patient Interface

Hospital, Specialties , & Doctor info
Doctor selection & appointment booking
Waitlist booking & cancellations
Case history & schedule viewing
Notifications: approvals, cancellations, updates
Prescription access

b) Doctor Interface

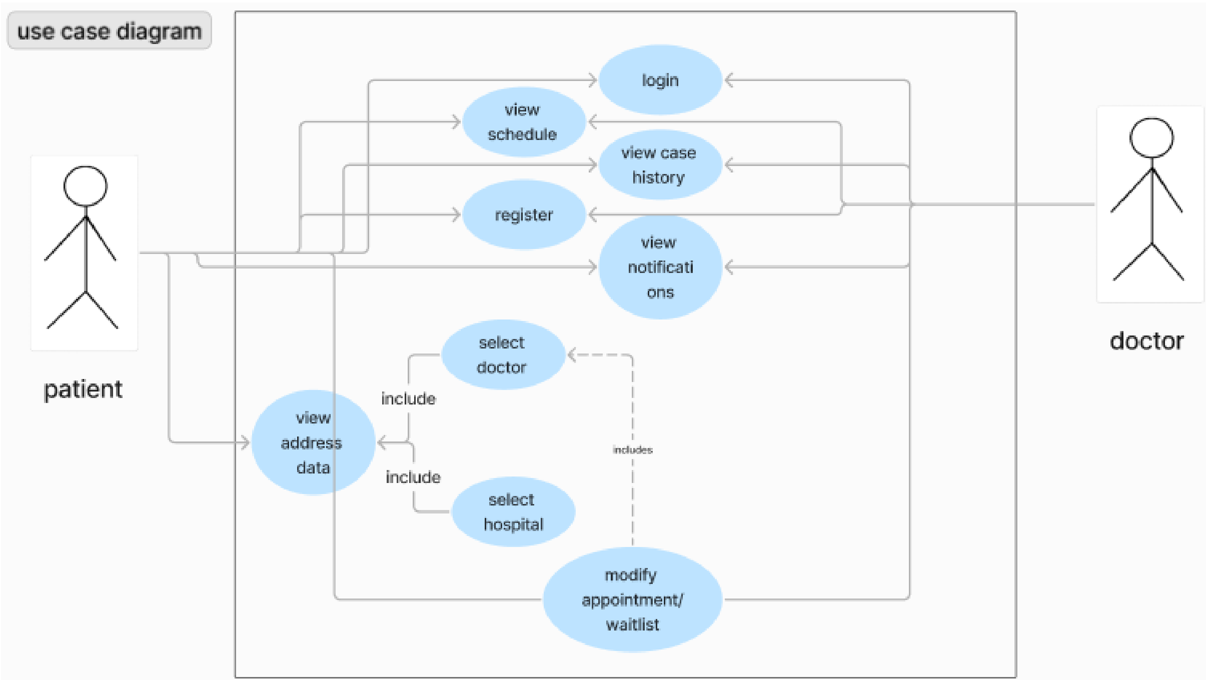
Appointment & waitlist management
Patient case history
Time slot allocation

c) Admin Interface

Full database & real-time access
Password embedded in server

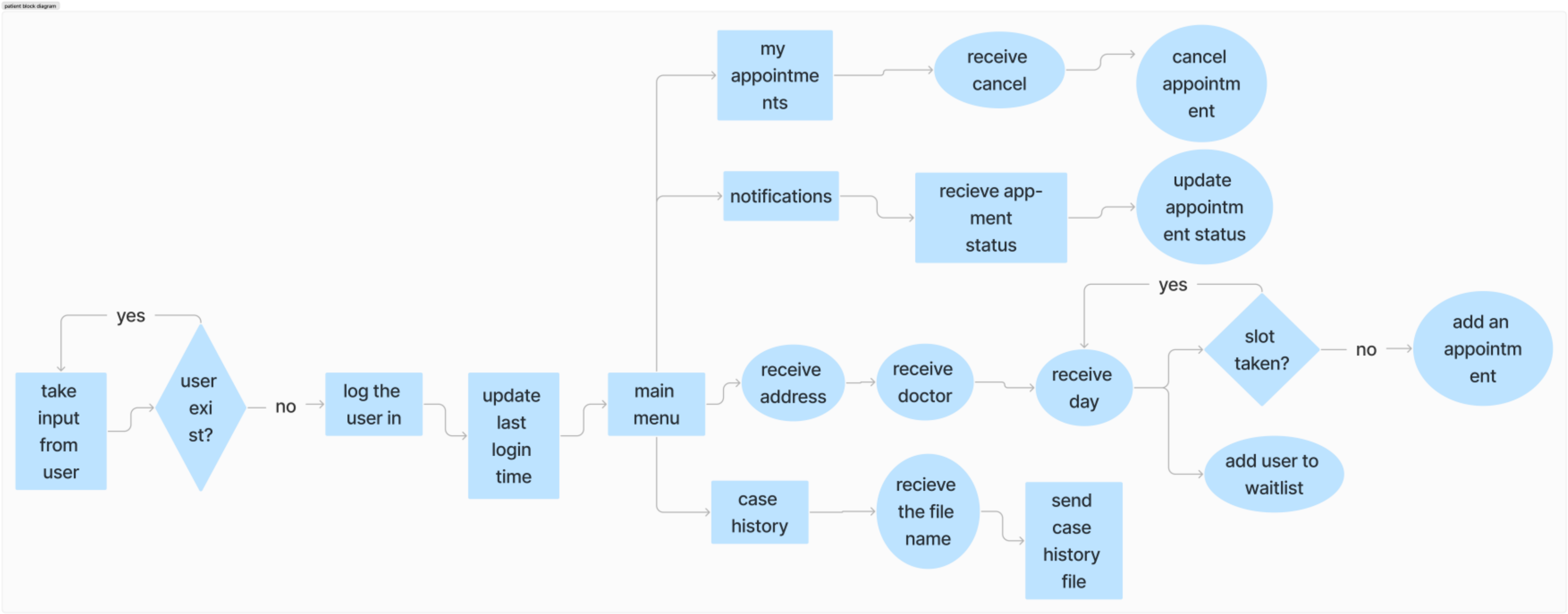
Requirement Definition

Use Case Diagram



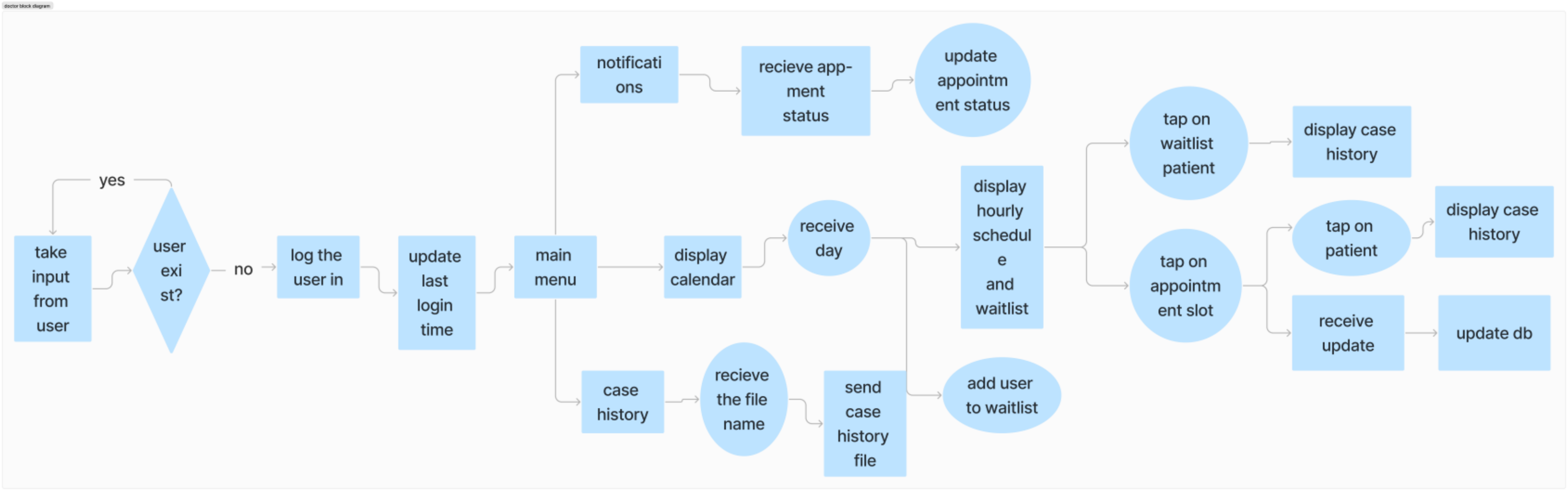
Requirement Definition

Activity Diagram for patient



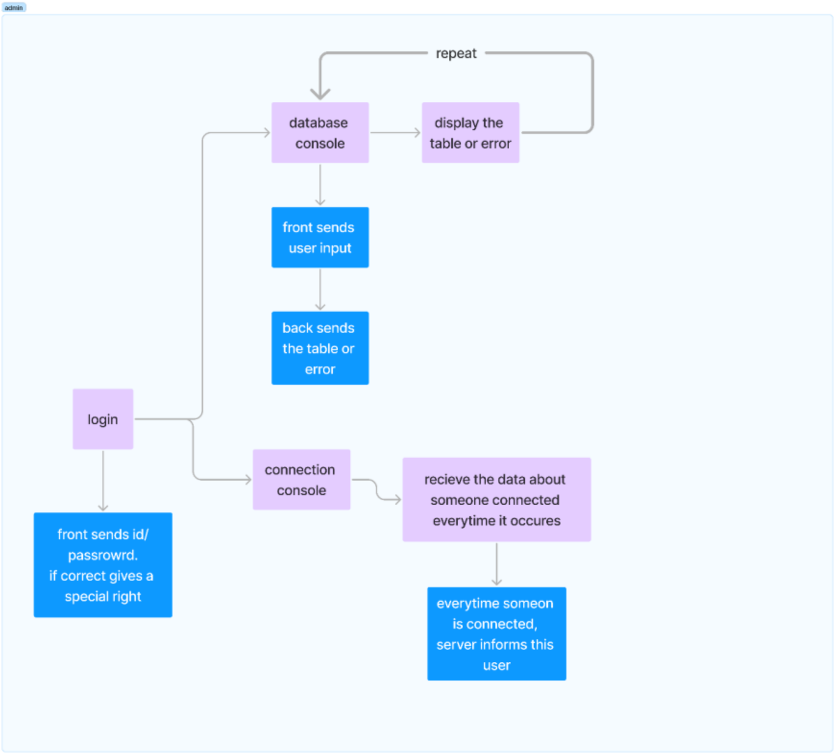
Requirement Definition

Activity Diagram for doctor



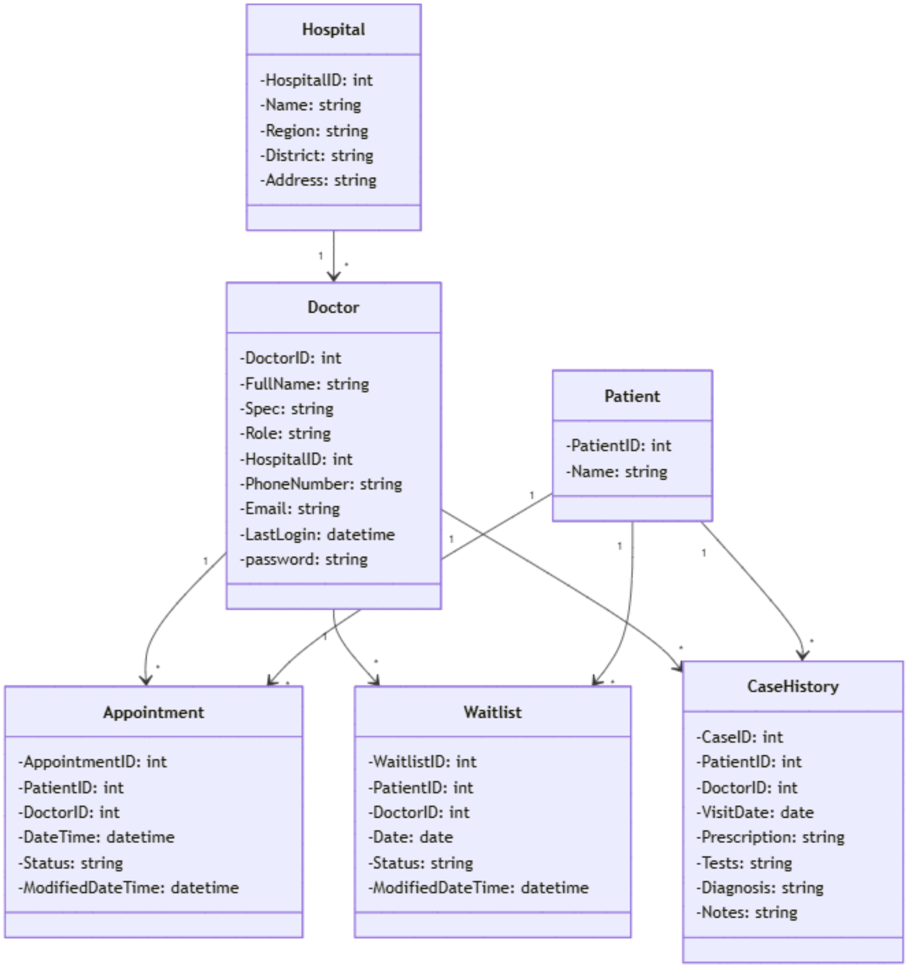
Requirement Definition

Activity Diagram for admin



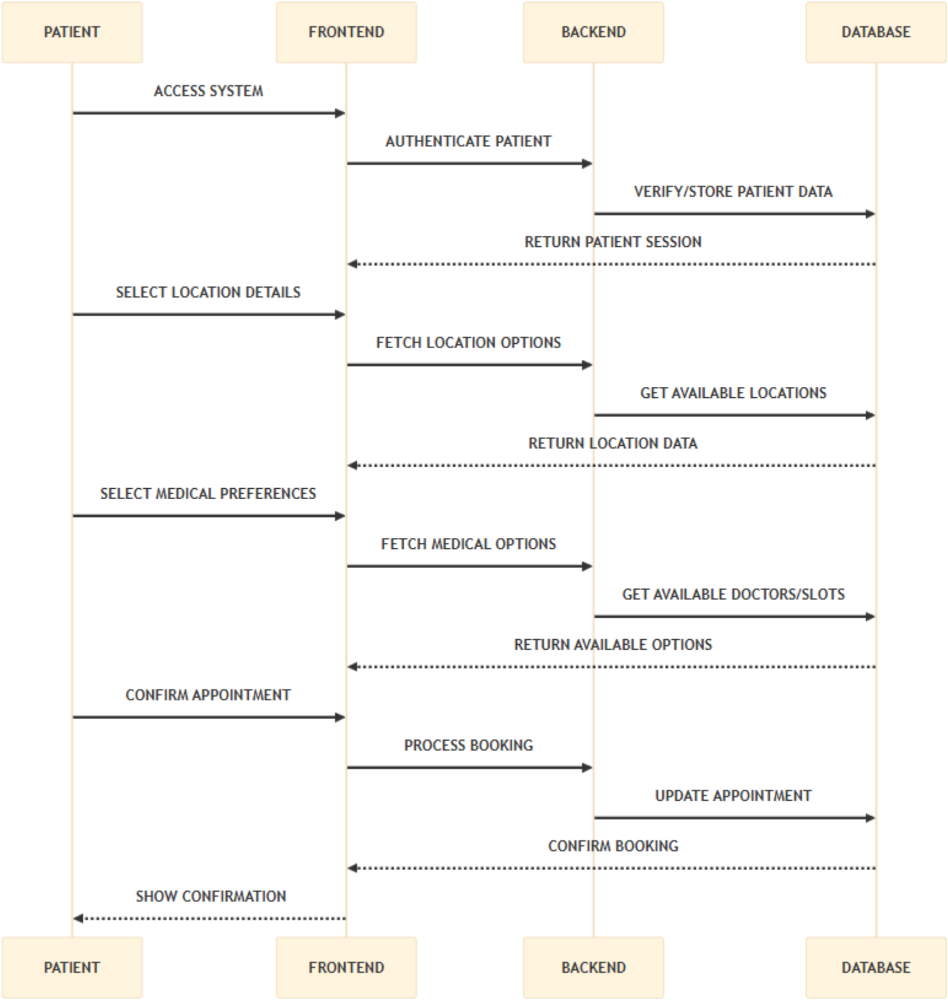
Requirement Definition

Class Diagram



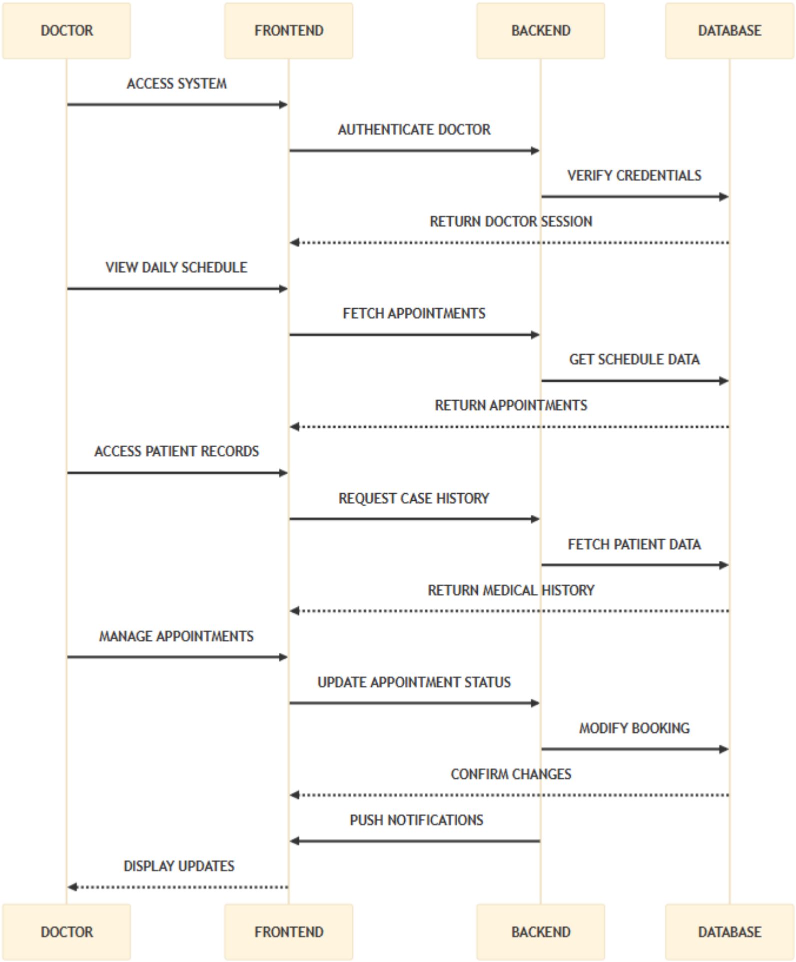
Requirement Definition

Behavioral (Sequential) Diagram for patients



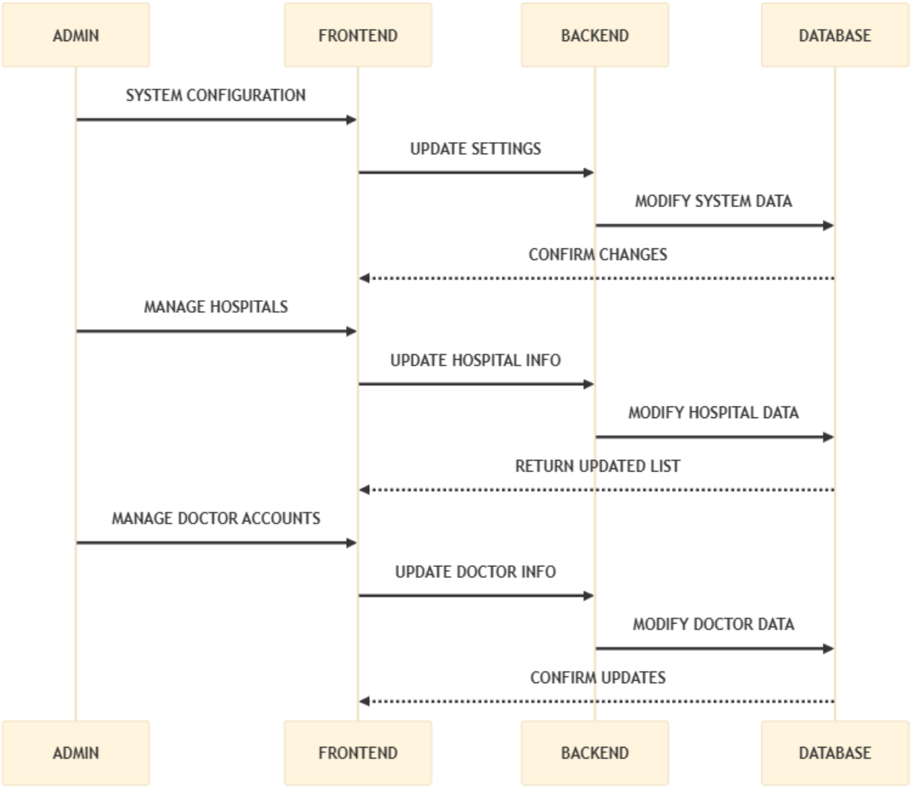
Requirement Definition

Behavioral (Sequential) Diagram for doctors



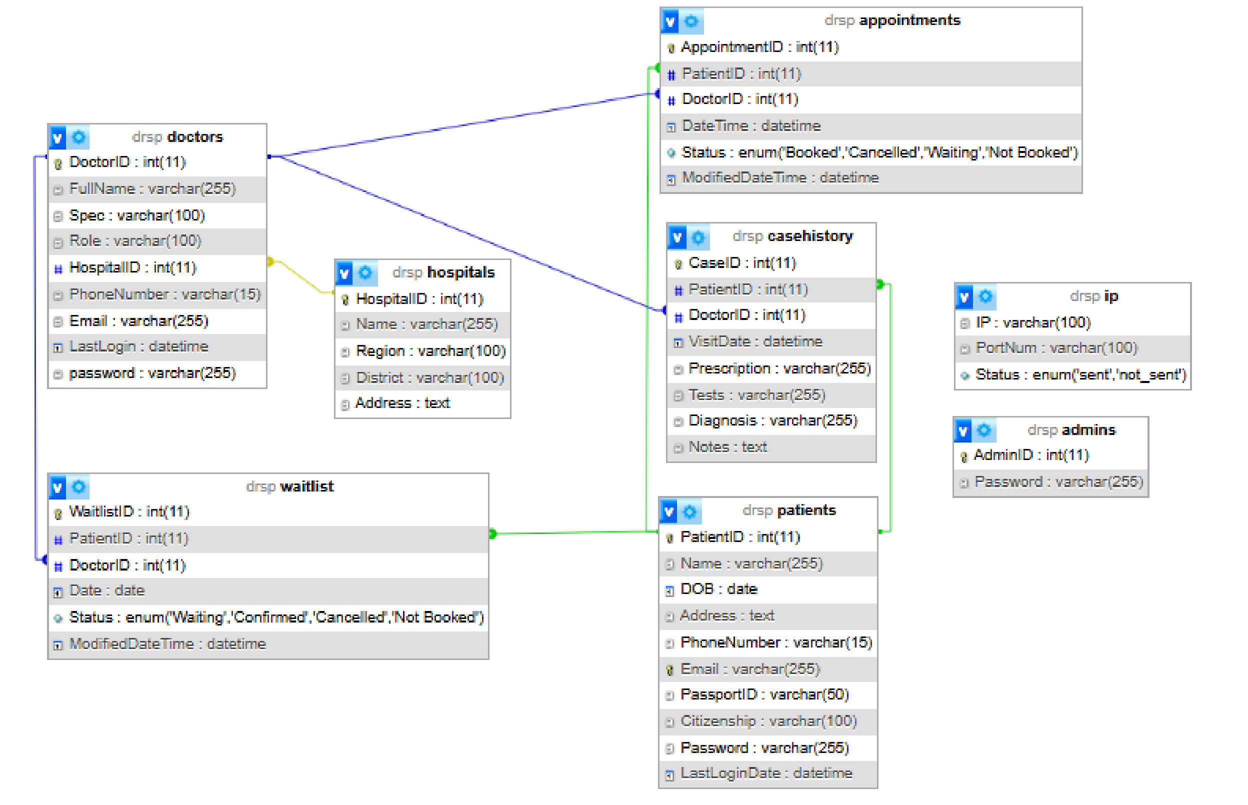
Requirement Definition

Behavioral (Sequential) Diagram for admin



Project design and implementation

Database design



Project design and implementation

Architecture

ElectronJS	GUI for desktop
C (sockets)	Communication between server and client
MySQL database	All the data is stored in mysql database

Project design and implementation

Front – end:

ElectronJS GUI, data input is transferred through shared object to C, converted to json and sent to server

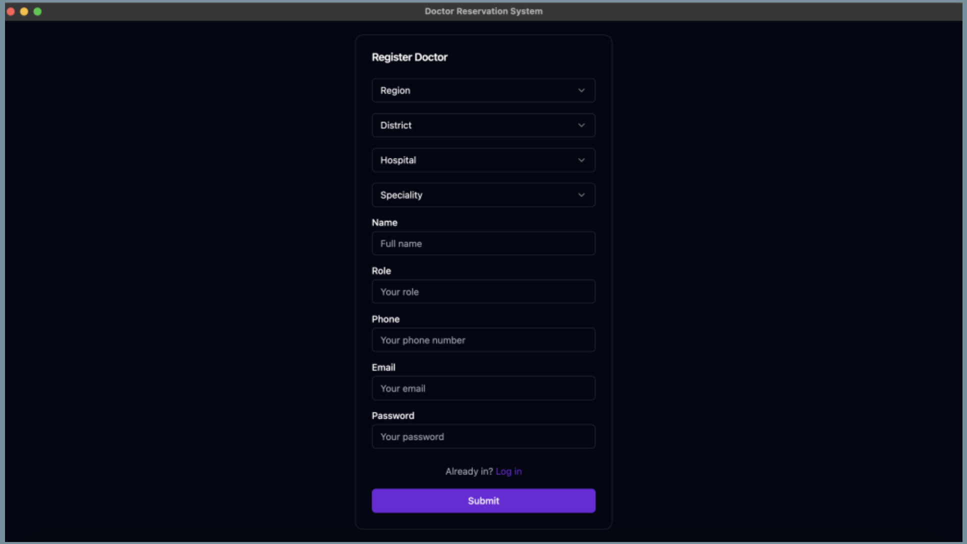
Middle – layer:

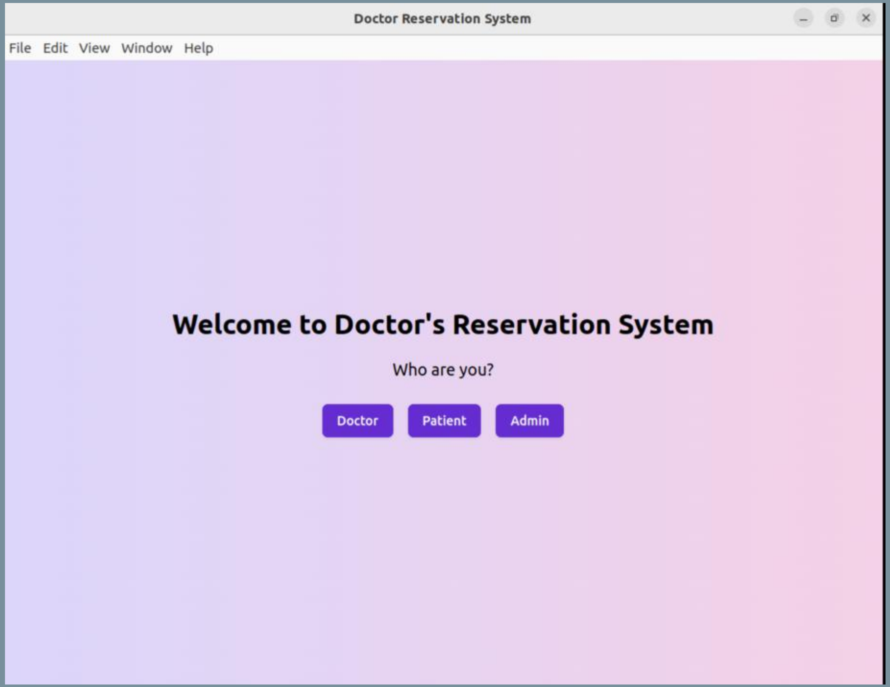
Open socket, transfer message from front to back, transfer message from back to front, close socket

Back – end:

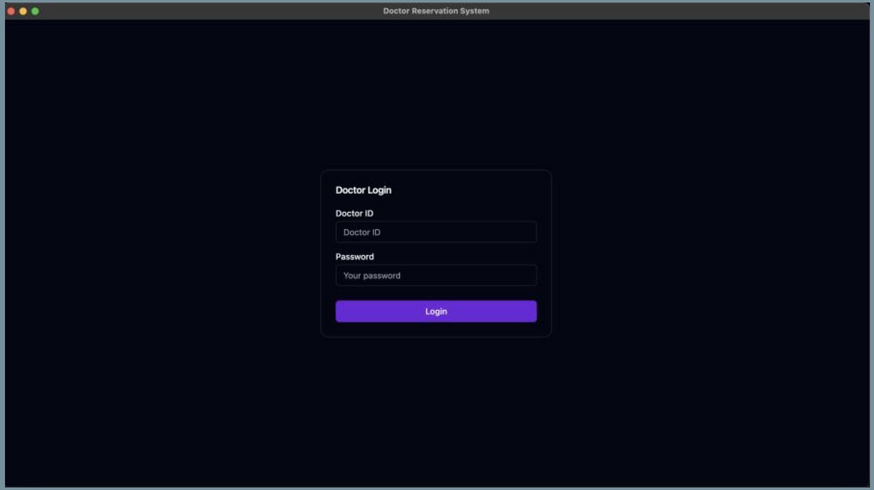
Receive json, de-parse it, execute it, convert to json using json-c, send

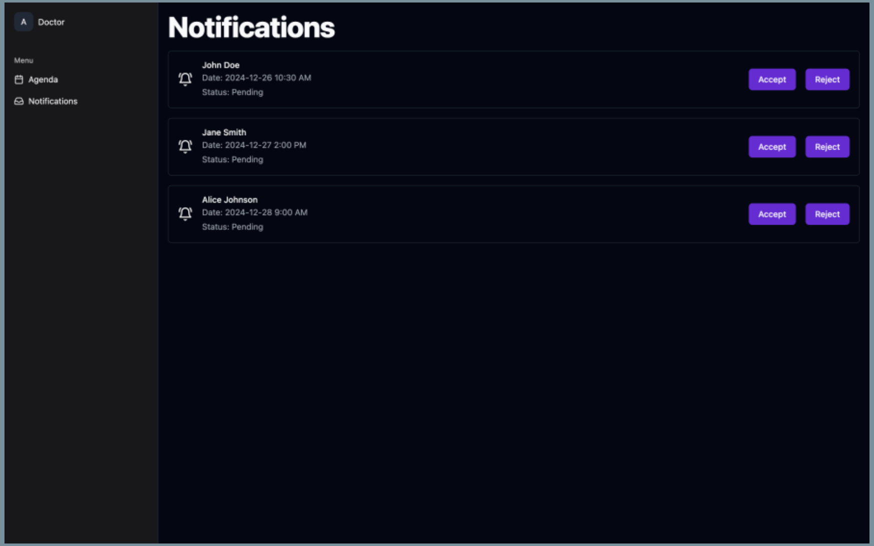
Results & Discussion patient



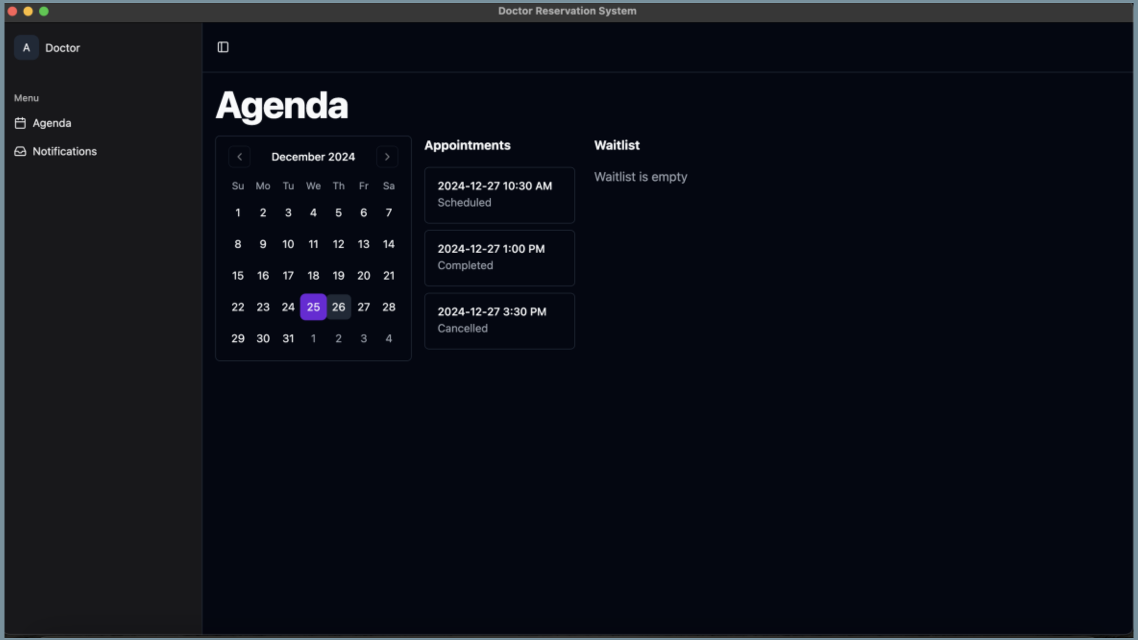


Results & Discussion patient

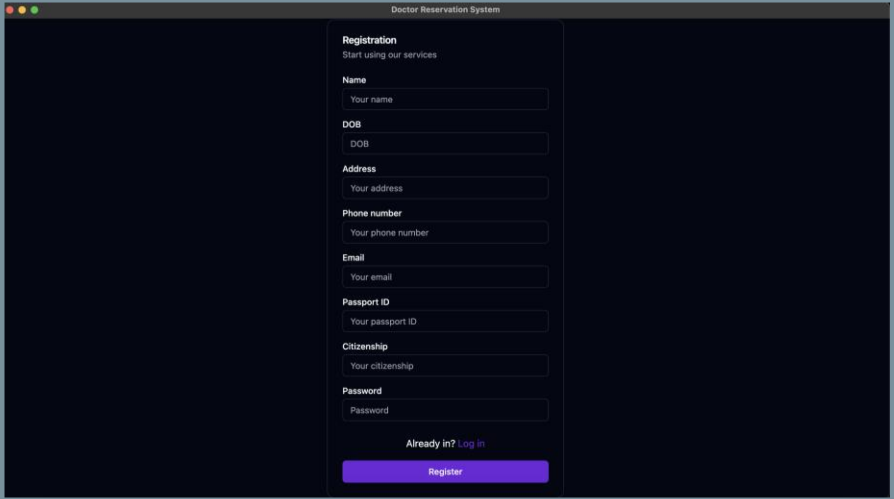


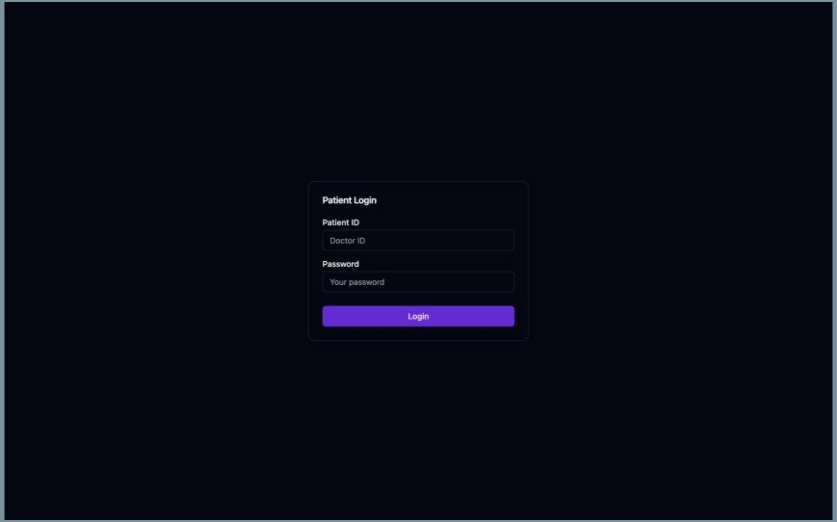


Results & Discussion patient

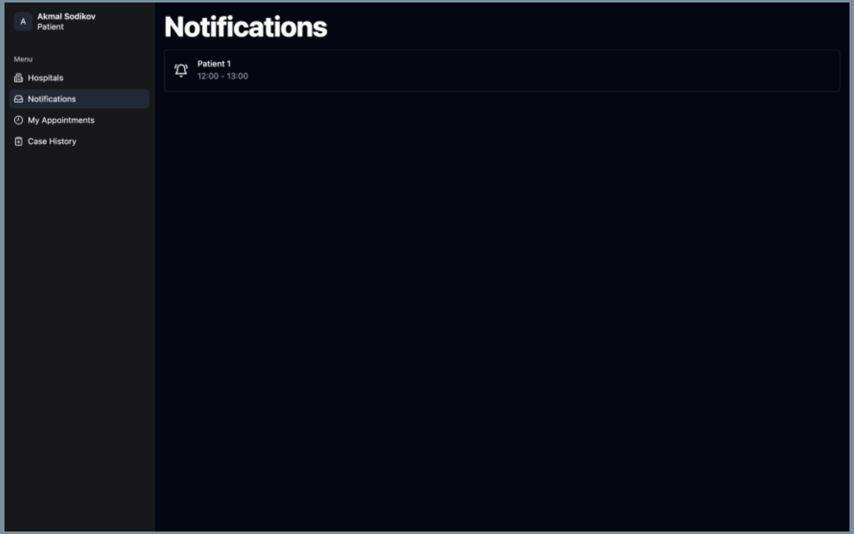


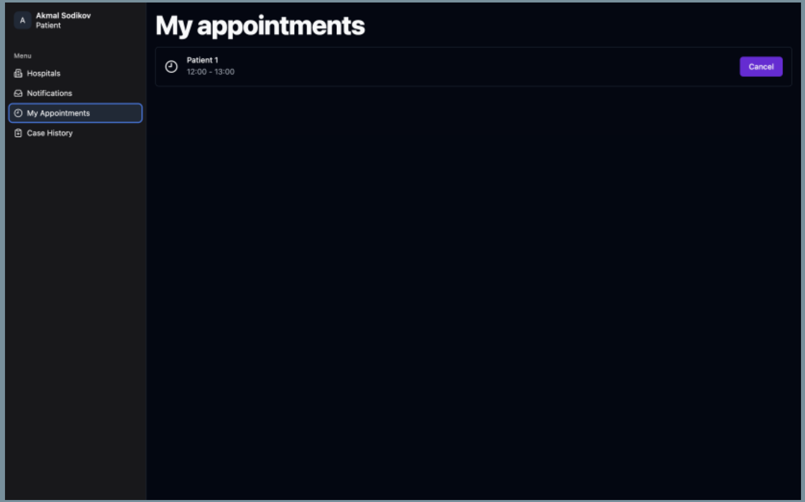
Results & Discussion doctor



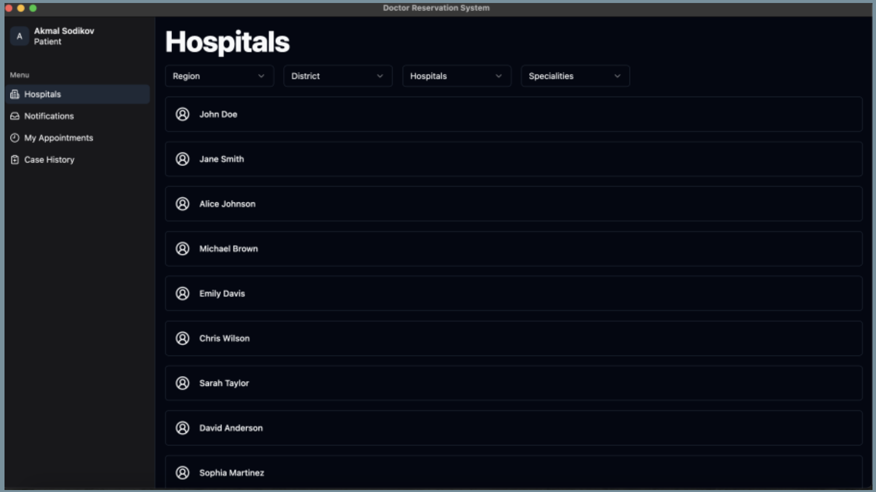


Results & Discussion doctor

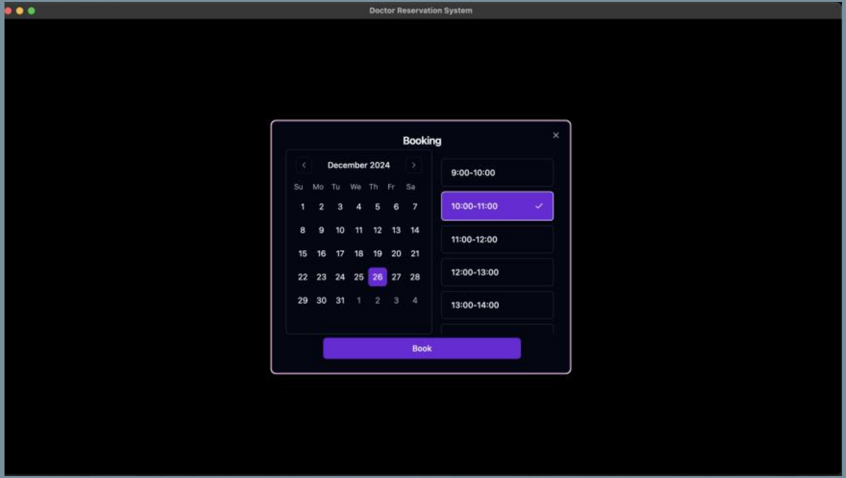




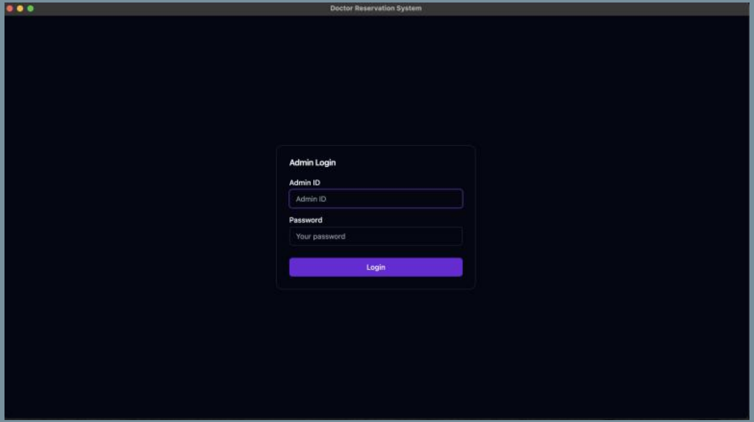
Results & Discussion doctor

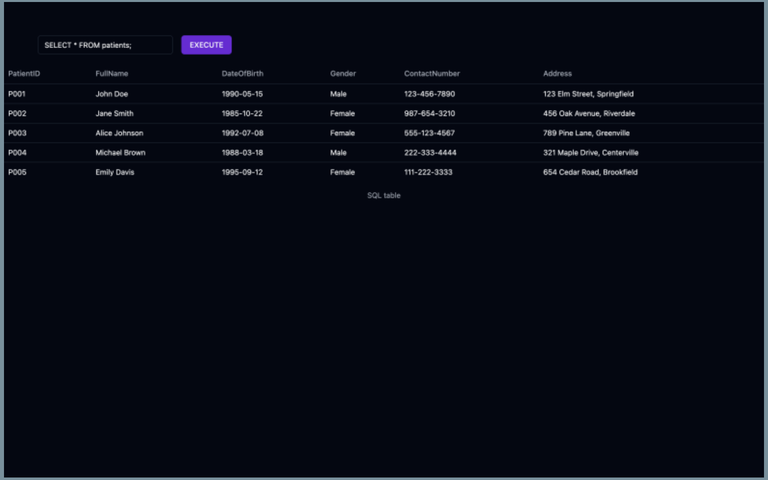


Results & Discussion doctor



Results & Discussion





PatientID	FullName	DateOfBirth	Gender	ContactNumber	Address
P001	John Doe	1990-05-15	Male	123-456-7890	123 Elm Street, Springfield
P002	Jane Smith	1985-10-22	Female	987-654-3210	456 Oak Avenue, Riverdale
P003	Alice Johnson	1992-07-08	Female	555-123-4567	789 Pine Lane, Greenville
P004	Michael Brown	1988-03-18	Male	222-333-4444	321 Maple Drive, Centerville
P005	Emily Davis	1995-09-12	Female	111-222-3333	654 Cedar Road, Brookfield

SQL table

Conclusion

Doctors Reservation System (DRSP)
Key Benefits:

1. **Efficiency**:
    1. Streamlined booking, real-time updates, and automated waitlists.
2. **User Experience**:
    1. Intuitive interface, reduced wait times, easy schedule management.
3. **Technical Innovation**:
    1. Secure, scalable architecture with multi-threaded processing.
4. **Healthcare Impact**:
    1. Better resource use, enhanced communication, improved care access, and digital records.

Future work

1. Enhance Security Features
2. System Scalability
3. More platforms
4. Go global

References

1 B. Hall, Beej’s Guide to Network Programming, 2020. [Online]. Available: https://beej.us/guide/bgnet/. [Accessed: Dec. 2, 2024].
2 W. R. Stevens, Unix Network Programming: The Sockets Networking API, 3rd ed., Boston, MA: Addison-Wesley, 2003.
3 A. Arora, "Socket programming in C with practical examples," GeeksforGeeks, 2021. [Online]. Available: https://www.geeksforgeeks.org/socket-programming-cc/.[Accessed: Dec. 12, 2024].
4 Tutorialspoint, "Linux socket programming tutorial," 2024. [Online]. Available: https://www.tutorialspoint.com/unix_sockets/unix_sockets_programming. [Accessed: Dec. 4, 2024].
5 P. Smith, "C and MySQL for backend development," MySQL Tutorial, 2020. [Online]. Available: https://www.mysqltutorial.org/c-and-mysql-backend-development/.[Accessed: Dec. 7, 2024].